LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhammad Fakih Nabal

(2509106009)

Kelas A1‘25

PROGRAM STUDI INFORMATIKA

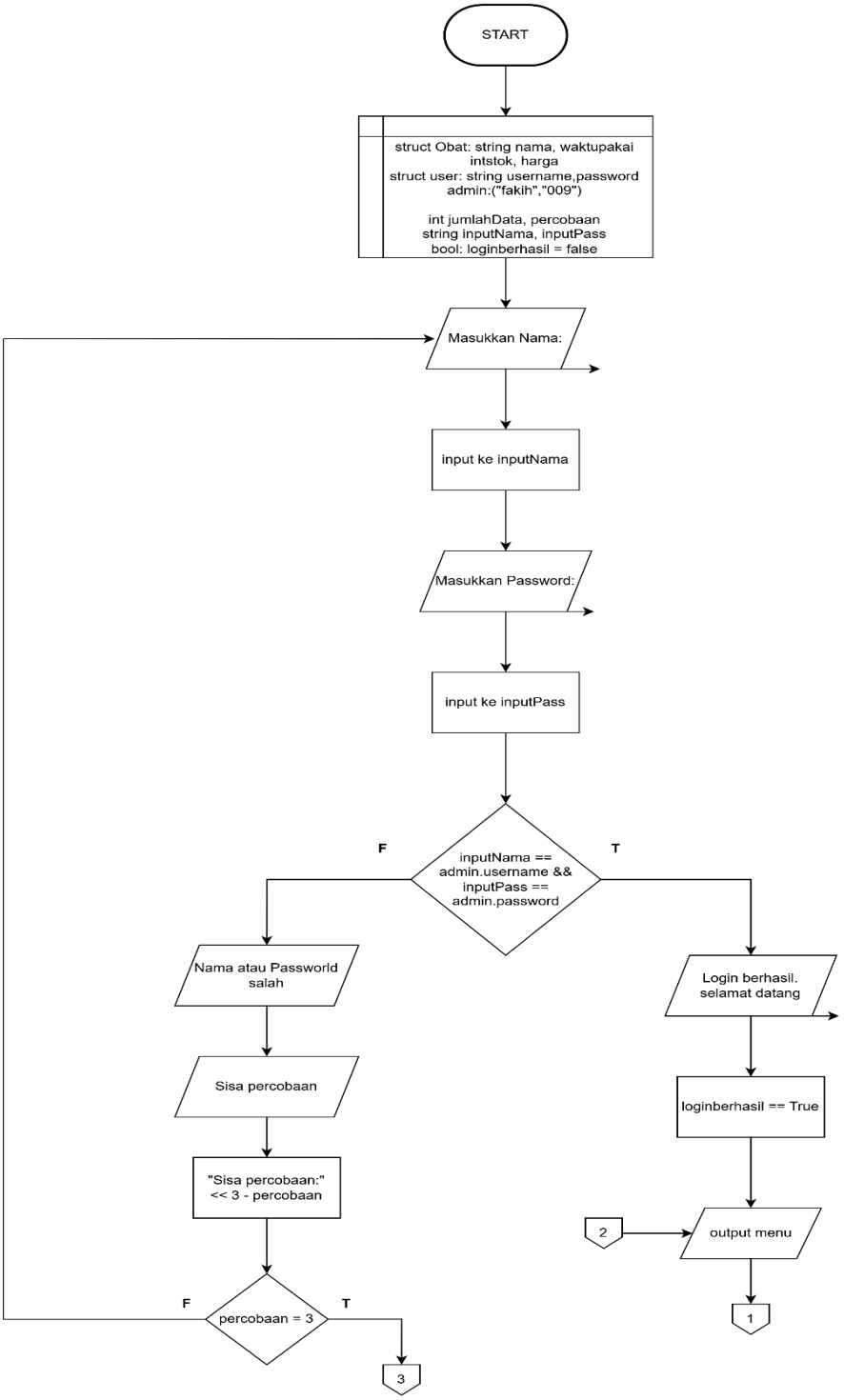
UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart

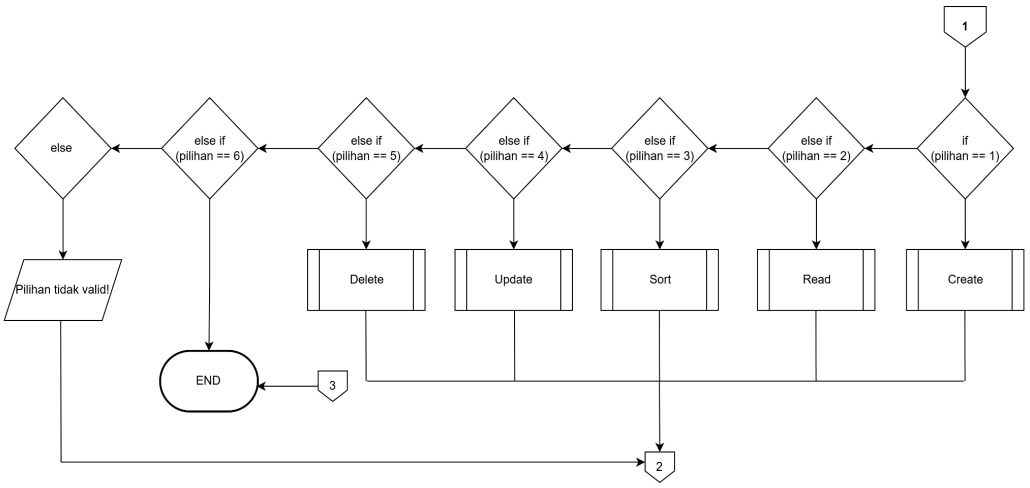
1. Menu utama



Gambar 1.1 flowchart

Program ini berawal dengan meminta user untuk melakukan login terlebih dahulu dengan menanyakan username dan password jika user menjawab dengan benar maka program akan membuka akses untuk melanjutkan program, jika user salah menjawab maka program akan kembali menanyakan username dan password, jika user salah menjawab sebanyak tiga kali maka program tidak akan memberikan akses ke user dan akan berhenti beroperasi.

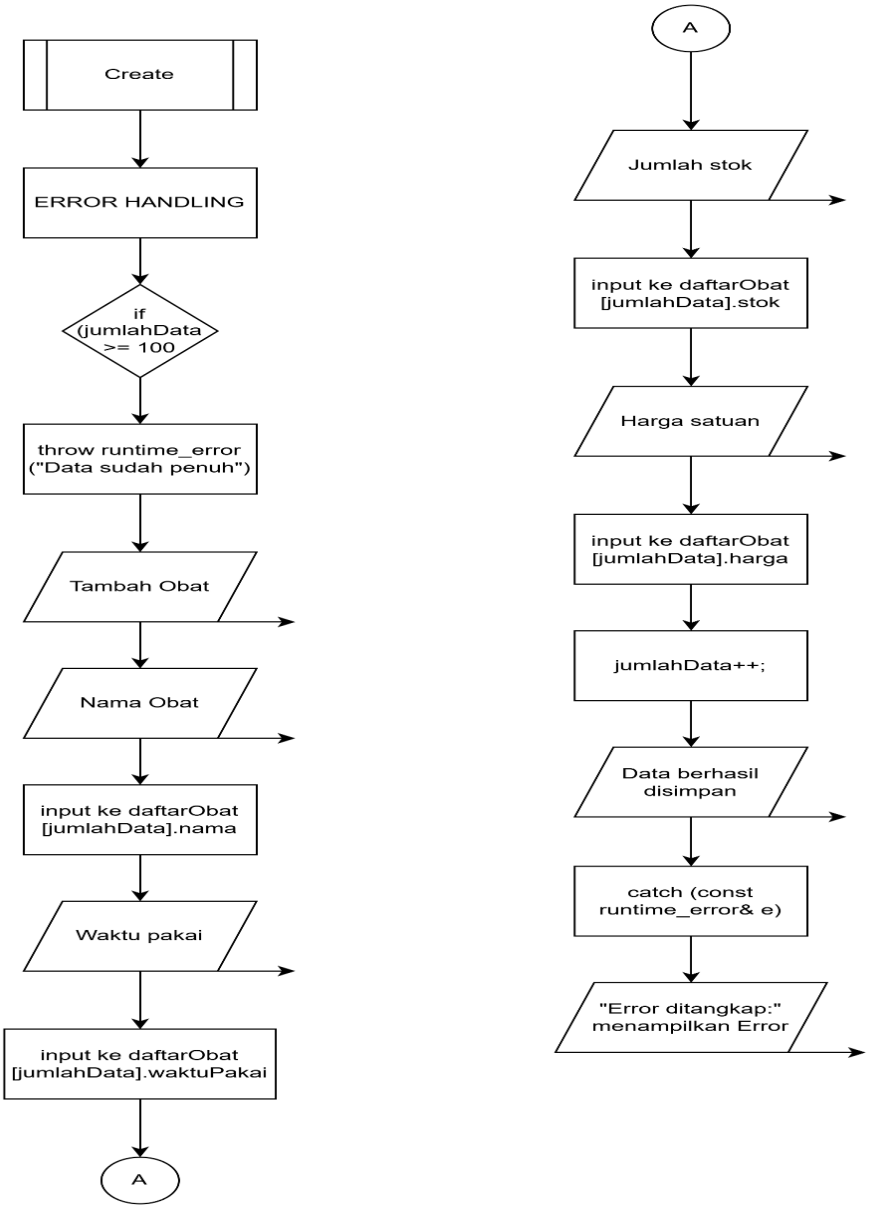
2. fungsi menu



Gambar 1.2 flowchart

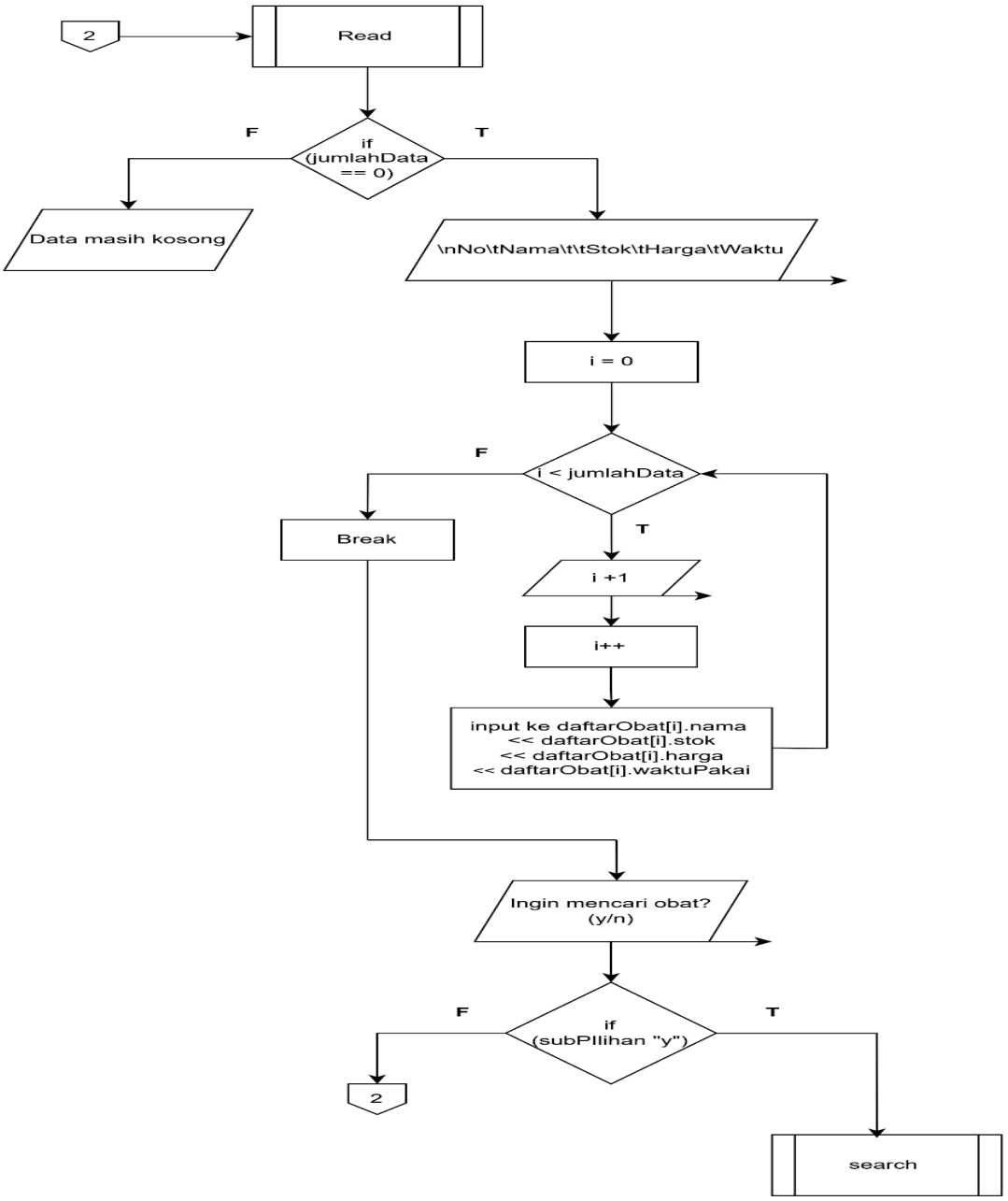
Setelah user menjawab dengan benar program akan menampilkan menu utama yang berisi 6 pilihan, pilihan pertama adalah “Tambah obat” disini user bisa menambahkan data obat baru. Pilihan kedua adalah “Tampilkan semua Obat” disini user bisa melihat data obat yang sudah ada, user juga bisa mencari data obat secara spesifik dengan memasukkan nama atau harga obat yang dicari. Pilihan ketiga adalah “Urutkan Obat” disini user bisa mengurutkan data yang sudah di input dari pilihan pertama menggunakan 3 metode pengurutan (sorting), yaitu Bubble sort, Selection sort, Insertion sort. Pilihan keempat adalah “Update Data” disini user bisa mengubah data yang sudah ada jika ada kekeliruan atau perubahan dari data yang sudah ada. Pilihan kelima adalah “Hapus Data” disini user bisa menghapus data obat yang sudah tidak diperlukan.

3. Create



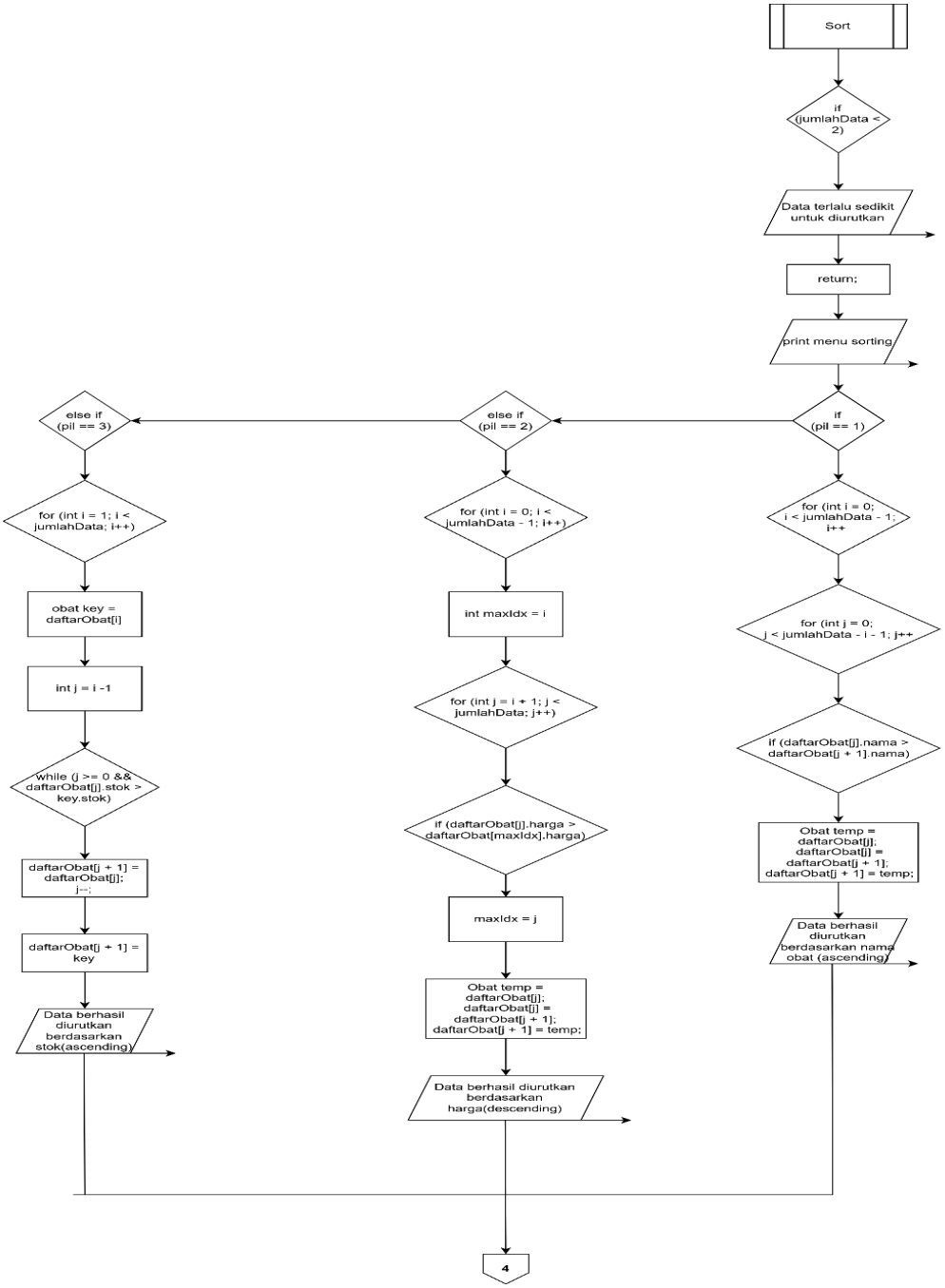
Gambar 1.3 flowchart

4. Read



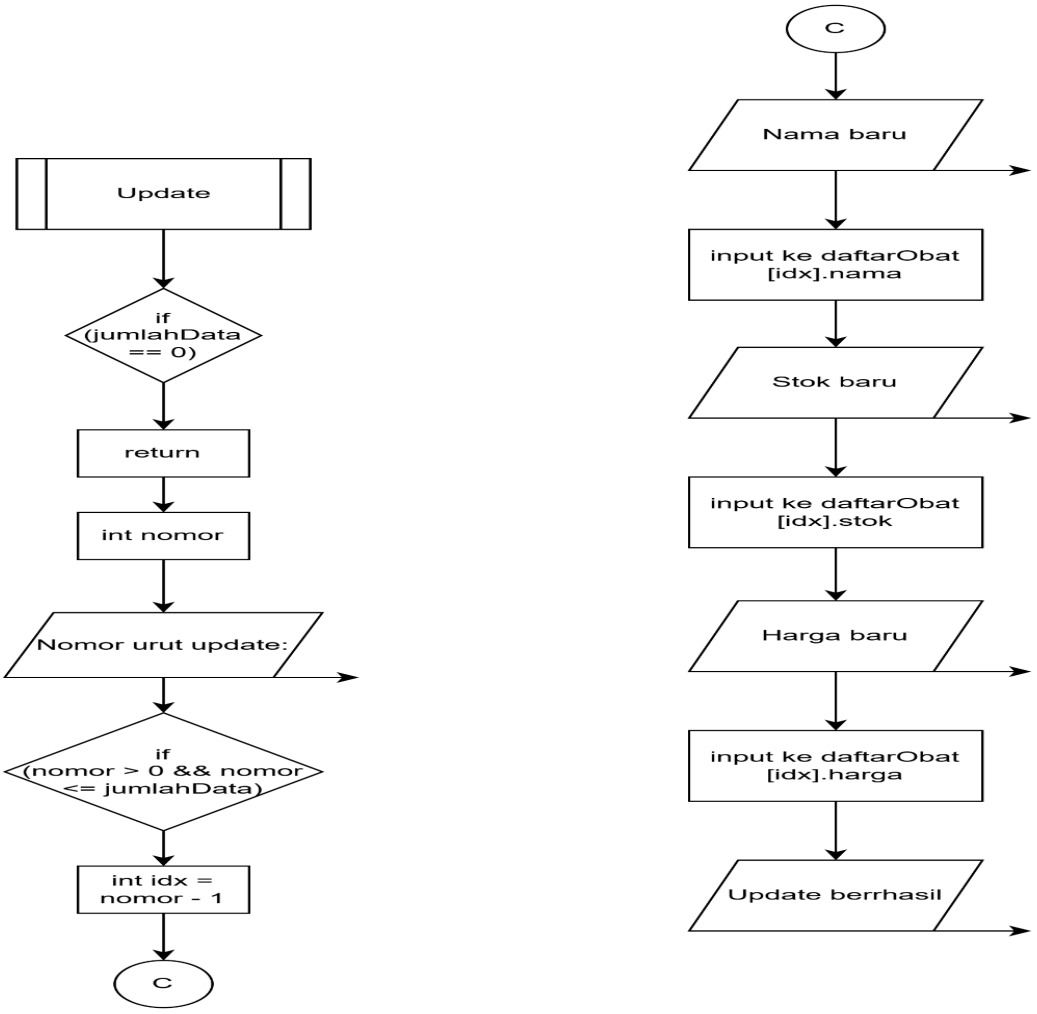
Gambar 1.4 flowchart

5. Sort



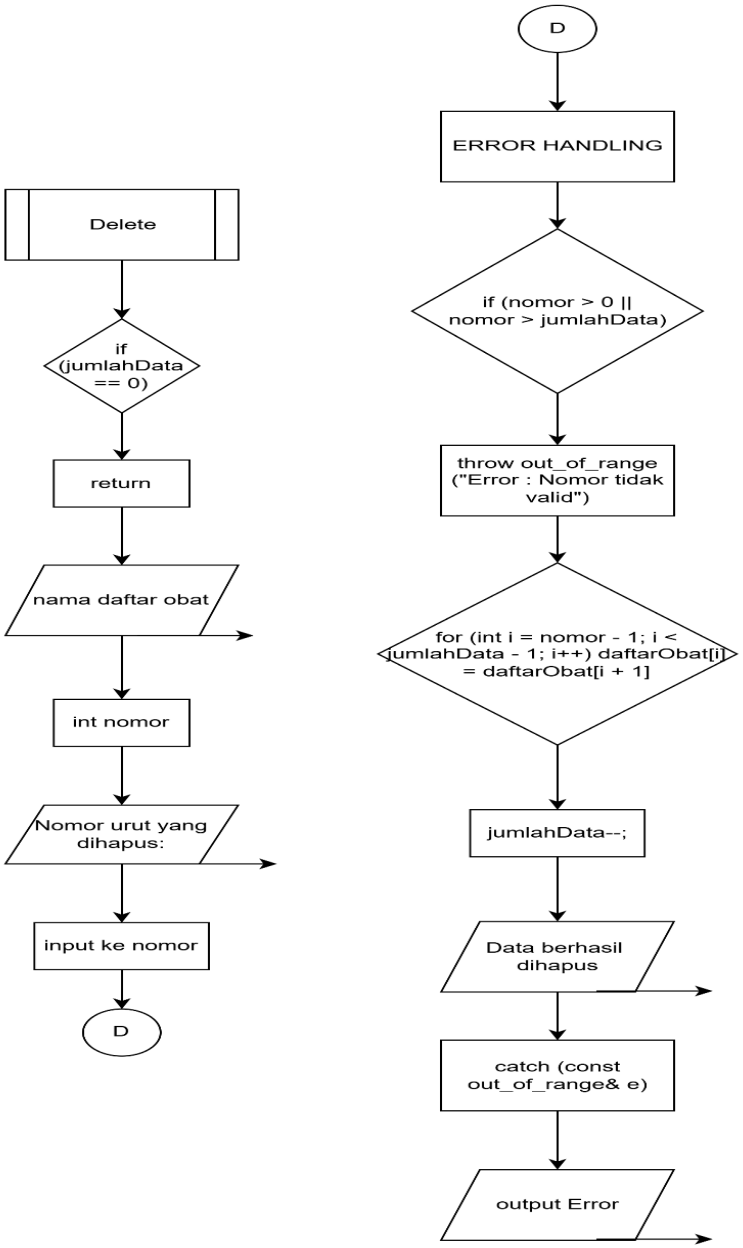
Gambar 1.5 flowchart

6. Update



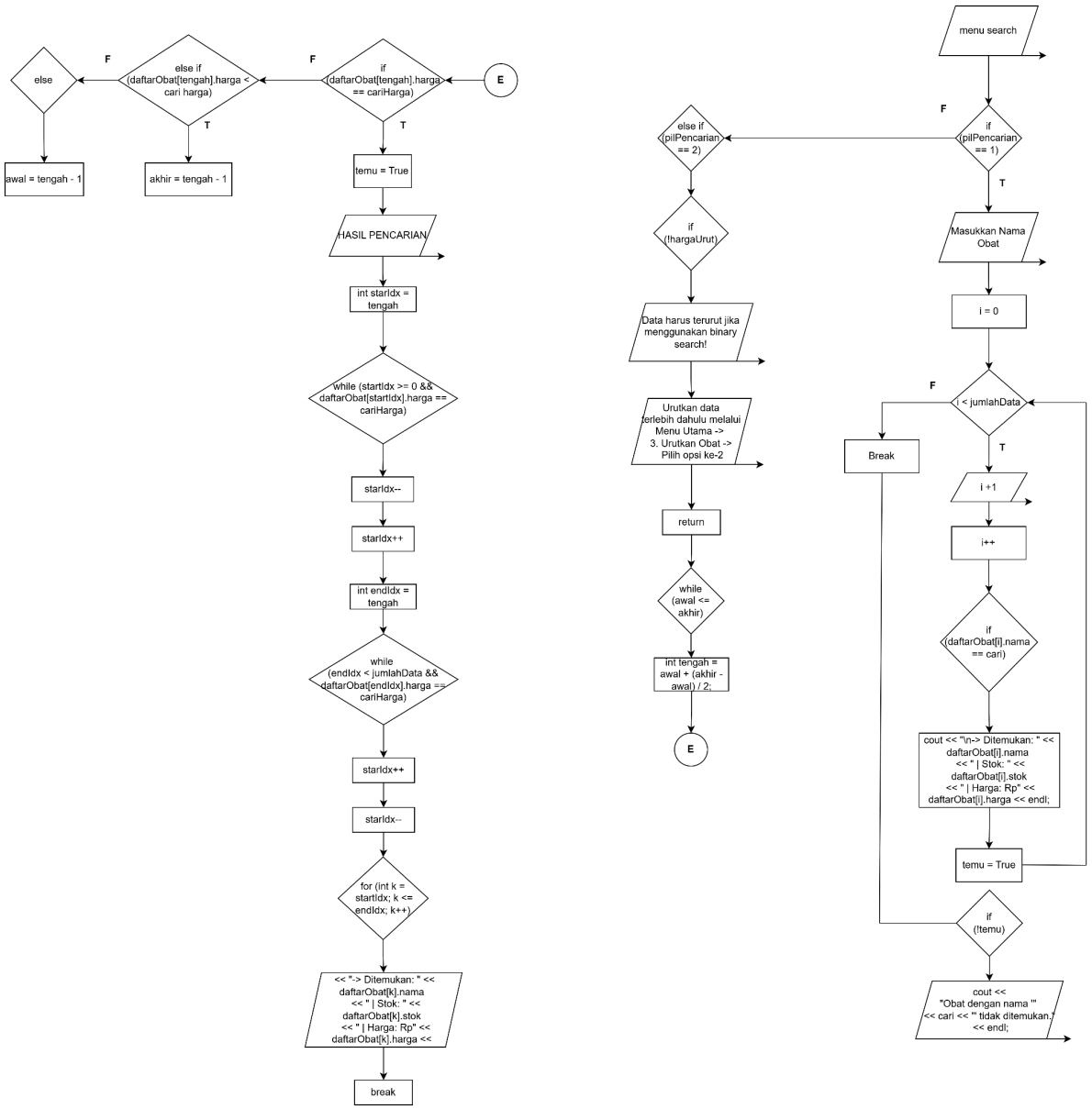
Gambar 1.6 flowchart

7. Delete



Gambar 1.7 flowchart

8. Search



Gambar 1.8 flowchart

2. Deskripsi Singkat Program

Program ini dibuat menggunakan metode pemrograman **predefined process** serta telah menerapkan konsep pointer **Address-of Operator** (&) dan **Dereference Operator** (*) yang bertujuan agar program lebih efisien dalam mengelola data. Program ini juga telah dilengkapi dengan metode **sorting** (Bubble, Selection, dan Insertion sort**), searching** (Linier dan Binary search) dan **ErrorHandling** yang berada di kode create dan delete. Program ini

bertujuan untuk memudahkan para apoteker untuk mengelola penyimpanan obat-obatan yang ada di apotik.

3. Source Code

Source Code:

```
void Create() {
    cout << "\n=== Tambah Obat ===" << endl;         // ERROR HANDLING
    try {
        if (jumlahData >= 100) {
    throw runtime_error("Data sudah penuh");
    }
    cin.ignore(1000, '\n');
    cout << "Nama obat      : "; getline(cin, daftarObat[jumlahData].nama);
    cout << "Waktu pakai  (pemakaian x jam)  : "; getline(cin, daftarObat[jumlahData].waktuPakai);
    cout << "Jumlah stok    : "; cin >> daftarObat[jumlahData].stok;
    cout << "Harga satuan  (angka) : "; cin >> daftarObat[jumlahData].harga;
    jumlahData++;
    hargaUrut = false;
    cout << "\nData berhasil disimpan" << endl;
}
    catch (const runtime_error& e) {
    cout << "Error ditangkap: " << e.what() << endl;
    }
}
```

Gambar 3.1 Create

```cpp
void Read() {
    if (jumlahData == 0) {
        cout << "\nData masih kosong" << endl;
    } else {
        cout << "\nNo\tNama\t\tWaktu\t\tStok\tHarga" << endl;
        cout << "------------------------------------------------------------" << endl;
        for (int i = 0; i < jumlahData; i++) {
            cout << i + 1 << "\t" << daftarObat[i].nama;
            if (daftarObat[i].nama.length() < 8) cout << "\t\t"; else cout << "\t";
            cout << daftarObat[i].waktuPakai;
            if (daftarObat[i].waktuPakai.length() < 8) cout << "\t\t"; else cout << "\t";
            cout << daftarObat[i].stok << "\t" << "Rp" << daftarObat[i].harga;
            if (daftarObat[i].stok < 5) cout << " (Stok hampir habis)";
            cout << endl;
        }
        char subPilihan;
        cout << "\nIngin mencari obat? (y/n): "; cin >> subPilihan;
        if (subPilihan == 'y' || subPilihan == 'Y') {
            Search();
        }
    }
}
```

Gambar 3.2 Read

```cpp
void Update() {
    if (jumlahData == 0) return;
        for (int i = 0; i < jumlahData; i++) cout << i + 1 << ". " << daftarObat[i].nama << endl;
        int nomor;
        cout << "\nNomor urut yang diupdate: "; cin >> nomor;
        cin.ignore(1000, '\n');
    cout << "Nomor urut update: "; cin >> nomor;
    if (nomor > 0 && nomor <= jumlahData) {
        int idx = nomor - 1;
        cin.ignore();
        cout << "Nama Baru: "; getline(cin, daftarObat[idx].nama);
        cout << "Stok Baru: "; cin >> daftarObat[idx].stok;
        cout << "Harga Baru: "; cin >> daftarObat[idx].harga;
        cout << "Update Berhasil" << endl;
    }
}
```

Gambar 3.3 Update

```
void Delete() {
    if (jumlahData == 0) {
        cout << "Data masih kosong" << endl;
        return;
    }
    cout <<  "\n=== DAFTAR OBAT ===" << endl;
    for (int i = 0; i < jumlahData; i++) {
        cout << i + 1 << ". " << daftarObat[i].nama << endl;
    }
    int nomor;
    cout << "\nNomor urut yang dihapus: "; cin >> nomor;
    try {                                                    // ERROR HANDLING
        if (nomor <= 0 || nomor > jumlahData) {
            throw out_of_range("Error: Nomor tidak valid");
        }
        for (int i = nomor - 1; i < jumlahData - 1; i++){
            daftarObat[i] = daftarObat[i + 1];
        }
        jumlahData--;
        cout << "Data berhasil dihapus" << endl;
    }
    catch (const out_of_range& e) {
        cout << e.what() << endl;
    }
    }
```

Gambar 3.4 Delete

```
int main() {
    User admin = {"fakih", "009"};
    if (!Login(admin)) {
        cout << "Kesempatan habis program berhenti" << endl;
        return 0;
    }
    int pilihan;
    do {
        Menu();
        if (!(cin >> pilihan)) {
            cout << "Input harus angka!" << endl;
            cin.clear();
            cin.ignore(1000, '\n');
            continue;
        }
        switch (pilihan) {
            case 1: Create(); break;
            case 2: Read(); break;
            case 3: Sort(); break;
            case 4: Update(); break;
            case 5: Delete(); break;
            case 6: cout << "Program Selesai" << endl; break;
            default: cout << "Pilihan tidak valid!" << endl;
        }
    } while (pilihan != 6);
    return 0;
}
```

Gambar 3.5 Main

```cpp
void Sort() {
    if (jumlahData < 2) {
        cout << "\nData terlalu sedikit untuk diurutkan" << endl;
        return;
    }
    cout << "\n=== MENU SORTING OBAT ===" << endl;
    cout << "1. Urutkan Nama (Ascending - Bubble Sort)" << endl;
    cout << "2. Urutkan Harga (Descending - Selection Sort)" << endl;
    cout << "3. Urutkan Stok (Ascending - Insertion Sort)" << endl;
    cout << "Pilih [1-3]: ";
    int pil; cin >> pil;
    if (pil == 1) {                                  // Bubble sort
        for (int i = 0; i < jumlahData - 1; i++) {
            for (int j = 0; j < jumlahData - i - 1; j++) {
                if (daftarObat[j].nama > daftarObat[j + 1].nama) {
                    Obat temp = daftarObat[j];
                    daftarObat[j] = daftarObat[j + 1];
                    daftarObat[j + 1] = temp;
                }
            }
        }
        cout << "Data berhasil diurutkan berdasarkan nama obat(ascending)" << endl;
        Read();
    }
    else if (pil == 2) {                             // Selection sort
        for (int i = 0; i < jumlahData - 1; i++) {
            int maxIdx = i;
            for (int j = i + 1; j < jumlahData; j++) {
                if (daftarObat[j].harga > daftarObat[maxIdx].harga) {
                    maxIdx = j;
                }
            }
            Obat temp = daftarObat[i];
            daftarObat[i] = daftarObat[maxIdx];
            daftarObat[maxIdx] = temp;
        }
        cout << "Data berhasil dirutkan berdasarkan harga(descending)" << endl;
        Read();
    } else if (pil == 3) {                           // Insertion sort
        for (int i = 1; i < jumlahData; i++) {
            Obat key = daftarObat[i];
            int j = i - 1;
            while (j >= 0 && daftarObat[j].stok > key.stok) {
                daftarObat[j + 1] = daftarObat[j];
                j--;
            }
            daftarObat[j + 1] = key;
        }
        cout << "Data berhasil diurutkan berdasarkan Stok(ascending)" << endl;
        Read();
    } else {
        cout << "Pilihan tidak valid" << endl;
    }
}
```

Gambar 3.6 Sort

```cpp
void Search() {
    if (jumlahData == 0) {
        cout << "Data masih kosong!" << endl;
        return;
    }
    int pilPencarian;
    cout << "\n=== METODE PENCARIAN ===" << endl;
    cout << "1. Cari berdasarkan Nama Obat (Linear Search)" << endl;
    cout << "2. Cari berdasarkan Harga Obat (Binary Search)" << endl;
    cout << "Pilih [1-2]: ";
    cin >> pilPencarian;
    if (pilPencarian == 1) {                                                    // Linear search
        cin.ignore(1000, '\n');
        string cari;
        cout << "Masukkan Nama Obat: "; getline(cin, cari);
        bool temu = false;
        for (int i = 0; i < jumlahData; i++) {
            if (daftarObat[i].nama == cari) {
                cout << "\n-> Ditemukan: " << daftarObat[i].nama
                    << " | Stok: " << daftarObat[i].stok
                    << " | Harga: Rp" << daftarObat[i].harga << endl;
                temu = true;
            }
        }
        if (!temu) cout << "Obat dengan nama '" << cari << "' tidak ditemukan." << endl;

    } else if (pilPencarian == 2) {                                             // Binary search
        if (!hargaUrut) {
            cout << "\nData harus terurut jika menggunakan binary search" << endl;
            cout << "Urutkan data terlebih dahulu melalui Menu Utama -> 3. Urutkan Obat -> Pilih opsi ke-2" << endl;
            return;
        }
        int cariHarga;
        cout << "Masukkan Harga Obat yang dicari: Rp"; cin >> cariHarga;
        int awal = 0;
        int akhir = jumlahData - 1;
        bool temu = false;
        while (awal <= akhir) {
            int tengah = awal + (akhir - awal) / 2;
            if (daftarObat[tengah].harga == cariHarga) {
                temu = true;
                cout << "\n=== HASIL PENCARIAN ===" << endl;
                int startIdx = tengah;
                while (startIdx >= 0 && daftarObat[startIdx].harga == cariHarga) {
                    startIdx--;
                }
                startIdx++;
                int endIdx = tengah;
                while (endIdx < jumlahData && daftarObat[endIdx].harga == cariHarga) {
                    endIdx++;
                }
                endIdx--;
                for (int k = startIdx; k <= endIdx; k++) {
                    cout << "-> Ditemukan: " << daftarObat[k].nama
                        << " | Stok: " << daftarObat[k].stok
                        << " | Harga: Rp" << daftarObat[k].harga << endl;
                }
                break;
            }
            else if (daftarObat[tengah].harga < cariHarga) {
                akhir = tengah - 1;
            }
            else {
                awal = tengah + 1;
            }
        }
        if (!temu) cout << "Obat dengan harga Rp" << cariHarga << " tidak ditemukan" << endl;
    } else {
        cout << "Pilihan tidak valid" << endl;
    }
}
```

Gambar 3.6 Search

4. Hasil Output

```
=====================================
    SISTEM MANAJEMEN OBAT APOTEK
=====================================
1. Tambah Obat
2. Tampilkan Semua Obat
3. Urutkan Obat
4. Update Data
5. Hapus Data
6. Keluar
Pilih menu [1-6]:
```

Gambar 4.1 Output menu

```
=== Tambah Obat ===
Nama obat       : Obat A
Waktu pakai  (pemakaian x jam)  : 4x1
Jumlah stok     : 12
Harga satuan  (angka) : 12000

Data berhasil disimpan
```

Gambar 4.2 Output pilihan 1

```
No      Nama            Waktu           Stok    Harga
-------------------------------------------------------------
1       paracetamol     3x24            16      Rp10000
2       paramex         2x24            6       Rp8000

 Ingin mencari obat? (y/n): y
Masukkan Nama Obat: paramex
Ditemukan: paramex | Stok 6
```

Gambar 4.3 Output pilihan 2

```
=== MENU SORTING OBAT ===
1. Urutkan Nama (Ascending - Bubble Sort)
2. Urutkan Harga (Descending - Selection Sort)
3. Urutkan Stok (Ascending - Insertion Sort)
Pilih [1-3]: 3
Data berhasil diurutkan berdasarkan Stok(ascending)
```

Gambar 4.4 Output pilihan 3

```
=== MENU SORTING OBAT ===
1. Urutkan Nama (Ascending - Bubble Sort)
2. Urutkan Harga (Descending - Selection Sort)
3. Urutkan Stok (Ascending - Insertion Sort)
Pilih [1-3]: 1
Data berhasil diurutkan berdasarkan nama obat(ascending)

No      Nama            Stok    Harga   Waktu
------------------------------------------------------------
1       A               12      Rp12000 2x1
2       B               14      Rp2311  3x1
3       c               15      Rp8000  5x2
4       d               55      Rp30000 6x10
5       e               32      Rp20000 3x24

Ingin mencari obat? (y/n):
```

Gambar 4.5 Output Bubble sort

```
=== MENU SORTING OBAT ===
1. Urutkan Nama (Ascending - Bubble Sort)
2. Urutkan Harga (Descending - Selection Sort)
3. Urutkan Stok (Ascending - Insertion Sort)
Pilih [1-3]: 2
Data berhasil dirutkan berdasarkan harga(descending)

No      Nama            Stok    Harga   Waktu
------------------------------------------------------------
1       d               55      Rp30000 6x10
2       e               32      Rp20000 3x24
3       A               12      Rp12000 2x1
4       c               15      Rp8000  5x2
5       B               14      Rp2311  3x1
```

Gambar 4.6 Output Selection sort

```
=== MENU SORTING OBAT ===
1. Urutkan Nama (Ascending - Bubble Sort)
2. Urutkan Harga (Descending - Selection Sort)
3. Urutkan Stok (Ascending - Insertion Sort)
Pilih [1-3]: 3
Data berhasil diurutkan berdasarkan Stok(ascending)

No      Nama            Stok    Harga   Waktu
------------------------------------------------------------
1       A               12      Rp12000 2x1
2       B               14      Rp2311  3x1
3       c               15      Rp8000  5x2
4       e               32      Rp20000 3x24
5       d               55      Rp30000 6x10
```

Gambar 4.7 Output Insertion sort

```
=== METODE PENCARIAN ===
1. Cari berdasarkan Nama Obat (Linear Search)
2. Cari berdasarkan Harga Obat (Binary Search)
Pilih [1-2]: 1
Masukkan Nama Obat: Paracetamol

-> Ditemukan: Paracetamol | Stok: 50 | Harga: Rp5000
```

Gambar 4.8 Hasil Linier search

```
=== METODE PENCARIAN ===
1. Cari berdasarkan Nama Obat (Linear Search)
2. Cari berdasarkan Harga Obat (Binary Search)
Pilih [1-2]: 2
Masukkan Harga Obat yang dicari: Rp10000

=== HASIL PENCARIAN ===
-> Ditemukan: Vitamin C | Stok: 100 | Harga: Rp10000
```

Gambar 4.8 Hasil Binary search

```
Pilih menu [1-5]: 3
1. paracetamol
2. paramex

Nomor urut yang diupdate: 2
Masukkan Nama Baru: promagh
Masukkan Stok Baru: 42
Masukkan Harga Baru: 8000
 Update Berhasil!

=====================================
    SISTEM MANAJEMEN OBAT APOTEK
=====================================
1. Tambah Obat Baru
2. Lihat & Cari Stok
3. Update Data Obat
4. Hapus Data Obat
5. Keluar
Pilih menu [1-5]: 2
No      Nama            Waktu           Stok    Harga
-------------------------------------------------------------
1       paracetamol     3x24            16      Rp10000
2       promagh         2x24            42      Rp8000

 Ingin mencari obat? (y/n): n
```

Gambar 4.10 Output pilihan 4

```
Pilih menu [1-5]: 4
1. paracetamol
2. promagh

Nomor urut yang dihapus:1
Data telah terhapus

=====================================
    SISTEM MANAJEMEN OBAT APOTEK
=====================================
1. Tambah Obat Baru
2. Lihat & Cari Stok
3. Update Data Obat
4. Hapus Data Obat
5. Keluar
Pilih menu [1-5]: 2
No      Nama            Waktu           Stok    Harga
-------------------------------------------------------------
1       promagh         2x24            42      Rp8000

 Ingin mencari obat? (y/n):
```

Gambar 4.11 Output pilihan 5

5. Langkah-langkah GIT

5.1 GIT Init

```
PS C:\Users\Hasmiyah\.vscode\praktikum-apd\post-test\post-test-apd-2> git init
Initialized empty Git repository in C:/Users/Hasmiyah/.vscode/praktikum-apd/post-test/post-test-apd-2/.git/
PS C:\Users\Hasmiyah\.vscode\praktikum-apd\post-test\post-test-apd-2>
```

Digunakan untuk menginisiasi repository Git ke lokal.

5.2 GIT Add

```
C:\Users\Hasmiyah\.vscode\praktikum-apd\post-test>git add .
```

Digunakan untuk menambahkan perubahan file ke staging area.

5.3 GIT Commit

```
:\Users\Hasmiyah\.vscode\praktikum-apd\post-test>git commit -m "posttest 5"
[main 4474e35] posttest 5
 1 file changed, 134 insertions(+)
 create mode 100644 post-test/posttest_apd_5/2509106009_Muhammad_Fakih_Nabal_PT_5.py
```

Digunakan untuk menyimpan perubahan file yang sudah dimasukkan ke staging area.

5.4 GIT Remote

```
C:\Users\Hasmiyah\.vscode\praktikum-apd\post-test>git remote add origin https://github.com/fakih004/praktikum-apd.git
error: remote origin already exists.
```

Digunakan untuk menghubungkan repository lokal ke repository lain.

5.5 GIT Push

```
PS C:\Users\Hasmiyah\.vscode\praktikum-apd\post-test\post-test-apd-2> git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 390.74 KiB | 14.47 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/fakih004/praktikum-apd.git
   6ce701c..860c336  main -> main
```

Digunakan untuk mengirim commit repository lokal ke repository remote.